

The steps to configure your IDE or text editor to work with Mypy differ depending on which IDE or text editor you're using. You can find instructions online by searching for “<your IDE> Mypy configure,” “<your IDE> type hints setup,” or something similar. If all else fails, you can always run Mypy from the Command Prompt or Terminal window.

Telling Mypy to Ignore Code

You might write code that for whatever reason you don't want to receive type hint warnings about. To the static analysis tool, the line might appear to use the incorrect type, but it's actually fine when the program runs. You can suppress any type hint warnings by adding a `# type: ignore` comment to the end of the line. Here is an example:

```
def removeThreesAndFives(number: int) -> int:
    number = str(number) # type: ignore
    number = number.replace('3', '').replace('5', '') # type: ignore
    return int(number)
```

To remove all the 3 and 5 digits from the integer passed to `removeThreesAndFives()`, we temporarily set the integer number variable to a string. This causes the type checker to warn us about the first two lines in the function, so we add the `# type: ignore` type hints to these lines to suppress the type checker's warnings.

Use `# type: ignore` sparingly. Ignoring warnings from the type checker provides an opening for bugs to sneak into your code. You can almost certainly rewrite your code so the warnings don't occur. For example, if we create a new variable with `numberAsStr = str(number)` or replace all three lines with a single `return int(str(number.replace('3', '').replace('5', '')))` line of code, we can avoid reusing the `number` variable for multiple types. We wouldn't want to suppress the warning by changing the type hint for the parameter to `Union[int, str]`, because the parameter is meant to allow integers only.

Setting Type Hints for Multiple Types

Python's variables, parameters, and return values can have multiple data types. To accommodate this, you can specify type hints with multiple types